DESAFIO VERX · DOCUMENTAÇÃO DE ARQUITETURA

# Testes e Validação

## Estratégia de testes

A estratégia atual combina cobertura de regra de negócio com cobertura de comportamento ponta a ponta em memória.

Ela foi desenhada para responder a perguntas diferentes:

- a regra do domínio está correta;
- a API se comporta como esperado;
- o fluxo lógico entre ledger e saldo diário funciona;
- a documentação representa a verdade do código.

## Camadas de teste existentes

### Testes unitários

Arquivos relevantes:

- create-entry.use-case.spec.ts
- apply-entry-to-daily-balance.use-case.spec.ts
- money.spec.ts

Validam:

- criação de lançamento e persistência de outbox/idempotência;
- retorno idempotente ao repetir `requestId`;
- aplicação correta de evento de crédito;
- ignorar mensagem duplicada;
- comportamento do tipo monetário compartilhado.

### E2E por serviço

Arquivos relevantes:

- ledger-service.e2e-spec.ts
- daily-balance-service.e2e-spec.ts

Validam:

- criação e listagem de lançamentos;
- idempotência de POST repetido;

- consolidação do saldo após aplicação de eventos;
- erro `404` quando não existe saldo para o dia solicitado.

### Fluxo completo

Arquivo relevante:

- full-flow.e2e-spec.ts

Valida:

- criação de crédito e débito no ledger;
- publicação lógica da outbox;
- processamento do evento pelo saldo diário;
- resposta final correta da API de consulta.

## Evidência de validação executada

Os seguintes comandos foram executados com sucesso neste ambiente:

```
npm install
npm run build
npm test
npm run test:e2e
```

Resultado observado:

- build concluído sem erro;
- 3 suítes unitárias aprovadas;
- 3 suítes e2e aprovadas;
- 11 testes aprovados no total.

## Validação da infraestrutura local

Foi executado também:

```
docker compose up -d
```

Durante a validação, foi identificado um gap real:

- a imagem `softwaremill/elasticmq-native:1.5.10` não existia mais;
- o `docker-compose.yml` foi ajustado para `softwaremill/elasticmq-native`.

Esse ajuste melhora a confiabilidade da execução local e alinha o repositório com um estado executável atual.

## Cobertura atual da estratégia de testes

- regras centrais do fluxo de lançamento;
- idempotência na requisição;
- idempotência no consumo;
- consolidação do saldo diário;
- fluxo lógico entre os dois domínios;
- comportamento esperado das APIs principais.

## Lacunas de cobertura ainda abertas

- concorrência real do publisher da outbox;
- concorrência real do consumidor em infraestrutura compartilhada;
- testes automatizados com fila e banco reais dentro da suíte;
- cenários de indisponibilidade da mensageria;
- retry, reprocessamento e DLQ;
- medição formal de carga e latência.

## Matriz consolidada de gaps

| Gap | Impacto | Gravidade | Evidência | Tratamento atual | Resolução na evolução |
|---|---|---|---|---|---|
| Validação executável era inexistente no início desta revisão | reduzia confiança na entrega | Alta | `node_modules` ausente no início do trabalho | Corrigido agora por execução real | não se aplica |
| Publicação concorrente da outbox pode duplicar envio | risco de envio duplicado em múltiplas instâncias | Alta | `listPending` + `publish` + `markPublished` sem claim/lock | Documentado como limitação atual | claim/lock ou ownership do worker na Fase 2 |
| Idempotência concorrente do consumidor é simples | corrida pode depender da constraint do banco | Média | `exists(messageId)` seguido de `save(messageId)` | Documentado como desenho atual aceitável para o escopo | endurecimento do controle concorrente na Fase 2 |

| Gap | Impacto | Gravidade | Evidência | Tratamento atual | Resolução na evolução |
|---|---|---|---|---|---|
| Ausência de retry/DLQ formal | operação de falha ainda é básica | Média | poller apenas loga erro | Documentado como limitação atual | DLQ, retry e reprocessamento controlado na Fase 2 |
| Observabilidade ainda básica | não sustenta operação madura | Média | sem métricas/tracing/alertas | Documentado como cobertura básica | métricas, tracing, dashboards e SLOs na Fase 3 |
| Banco compartilhado entre contextos | reduz isolamento operacional | Média | schemas no mesmo PostgreSQL | Documentado como trade-off intencional | separação física de persistência na Fase 2 |
| Contrato de evento estava subdocumentado | dificultava leitura arquitetural | Baixa | contrato existia sem centralidade documental | Corrigido agora na documentação | governança mais forte do contrato na Fase 2 |
| NFRs sem classificação de maturidade | gerava ambiguidade | Média | sem matriz de maturidade | Corrigido agora na documentação | formalização operacional mais forte na Fase 3 |
| Fluxo arquitetural distribuído no código | dificultava leitura executiva | Baixa | narrativa espalhada em use cases e services | Corrigido agora na documentação | refinamento contínuo da narrativa arquitetural |

## Checklist executivo para futuras revisões

- instalar dependências;
- validar build;
- validar testes unitários;
- validar testes e2e;
- subir infraestrutura local;
- verificar aderência entre docs, controllers, use cases e contratos;
- confirmar que README, `docs/*.md`, `docs/drawio/*` e `docs/pdf/*` contam a mesma história;
- revisar se nenhum item futuro está descrito como implementado hoje.